

spd_reader Module

Design Report

SPD Fast-Path Detection via I²C EEPROM Read — DDR2 / DDR3 / DDR4 / DDR5

Contents

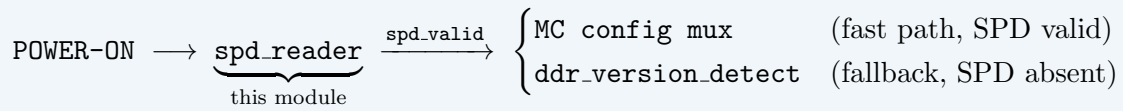
1	Purpose & Context	1
2	SPD Key Byte — Per-Generation Encoding	1
3	Module Interface	2
4	Block Diagram	2
5	Internal Block Descriptions	2
5.1	I ² C Bit Master	2
5.2	Byte Engine	2
5.3	SPD FSM	4
5.4	NACK Detector	4
5.5	Key Byte Decoder	4
5.6	Output Register	4
6	SPD FSM State Diagram	4
7	I²C Transaction Waveform	5
8	Timing & Latency Analysis	5
9	Integration with ddr_version_detect	6

§1 Purpose & Context

Every JEDEC DIMM carries a Serial Presence Detect (SPD) EEPROM on its PCB. The EEPROM is accessed over a two-wire I²C bus and contains factory-programmed fields that fully describe the installed memory, including its DDR generation, speed grade, geometry, and timing parameters.

`spd_reader` implements a minimal I²C master that reads the SPD *key byte* immediately after power-on reset. If the byte is valid, it asserts `spd_valid` and drives `ddr_version` to the Memory Controller (MC), allowing the training-based probe FSM (`ddr_version_detect`) to be *bypassed entirely*. Detection latency drops from thousands of training cycles to fewer than 100 I²C clock cycles.

Position in the MC Boot Sequence



§2 SPD Key Byte — Per-Generation Encoding

The JEDEC SPD specification defines a *key byte* at a fixed EEPROM address that identifies the DRAM generation. The address and encoding differ per generation, but the concept is identical across all four.

Field	DDR2	DDR3	DDR4	DDR5
JEDEC SPD spec	JESD21C	JESD79-3	JESD79-4	JESD79-5
Key byte address	0x02	0x02	0x02	0x01
Key byte value	0x08	0x0B	0x0C	0x12
I ² C device address	0x50+SA	0x50+SA	0x50+SA	0x50+SA
Page select (DDR5)	—	—	—	MR byte 0
Total SPD size	256 B	256 B	512 B	1024 B
Bytes needed for detect	3	3	3	2

Table 1: SPD key byte locations and values per DDR generation. SA = slot address pins (0–7) set by DIMM slot wiring.

The read operation requires only 2–3 bytes regardless of total SPD size, keeping the fast-path latency low.

§3 Module Interface

Port	Dir	Description
<code>clk</code>	in	System clock
<code>rst_n</code>	in	Active-low synchronous reset
<code>start</code>	in	Assert to begin SPD read; held until <code>done</code>
<code>slot_addr[2:0]</code>	in	DIMM slot address (SA2:SA0) wired from board
<code>scl</code>	out	I ² C clock (open-drain; needs external pull-up)
<code>sda</code>	inout	I ² C data (open-drain, bidirectional)
<code>spd_valid</code>	out	High when key byte was read and recognised
<code>ddr_version[2:0]</code>	out	0=DDR2, 1=DDR3, 2=DDR4, 3=DDR5
<code>done</code>	out	Strobe: read complete (valid or not)
<code>spd_absent</code>	out	High when NACK received — no SPD present
<code>raw_keybyte[7:0]</code>	out	Raw SPD byte 0x02 (debug / logging)

Table 2: Module port list. `sda` must be driven through an open-drain cell.

§4 Block Diagram

Figure 1 shows the internal structure of `spd_reader`. The I²C Master generates the physical SCL/SDA waveforms; the Byte Engine sequences address and register bytes; the Key Byte Decoder maps the raw value to a DDR generation; the Output Register holds results until the next reset.

§5 Internal Block Descriptions

5.1 I²C Bit Master

Generates the SCL clock and drives/samples SDA at the bit level. Implements the four I²C primitives: START condition, STOP condition, byte write (with ACK sample), and byte read (with ACK/NACK drive). SCL frequency is controlled by a configurable clock divider; a typical value of 400 kHz (Fast-Mode) keeps detection under 100 cycles at any system clock above 40 MHz.

Open-drain drive is mandatory — the module drives SDA low or releases it (high-Z); pull-up resistors on the board set the high level. The SCL line is similarly driven low or released.

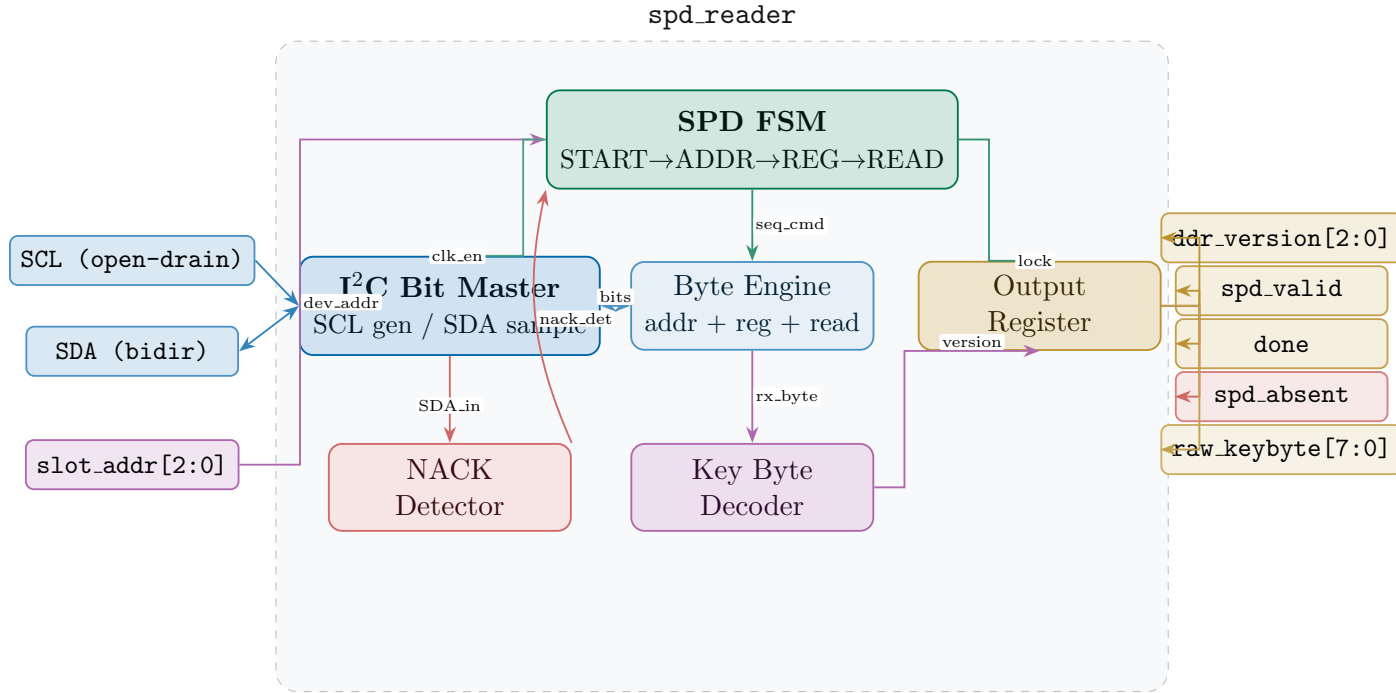


Figure 1: Top-level block diagram of `spd_reader`. **Blue**: I²C master/byte engine; **Green**: SPD FSM; **Purple**: key byte decoder; **Red**: NACK detector; **Amber**: output register.

5.2 Byte Engine

Sequences multi-byte I²C transactions commanded by the SPD FSM:

1. Write the 7-bit device address + W bit.
2. Write the register address byte (SPD byte offset).
3. Issue a REPEATED START.
4. Write the device address + R bit.
5. Read one data byte, driving NACK on the final byte to terminate the transfer.

Each step is driven by a `seq_cmd` token from the SPD FSM.

5.3 SPD FSM

The top-level sequencer. Drives the Byte Engine through the correct transaction for each generation probe attempt, handles retries on NACK, and asserts `lock` to the Output Register when a valid key byte is received.

See Section 6 for the full state diagram.

5.4 NACK Detector

Samples SDA during the ACK slot (9th clock pulse of each byte transfer). A high SDA indicates the target device did not pull the line low — NACK. Three consecutive NACKs on the device address trigger `spd_absent`, indicating no SPD EEPROM is present at the polled slot address.

A combinational lookup that maps the raw 8-bit SPD key byte to a 3-bit DDR generation code:

Table 3: Key byte decoder truth table. Unknown values assert the fallback path.

5.6 Output Register

§6 SPD FSM State Diagram

```

graph LR
    RESET((RESET)) -- "rst_n=1" --> IDLE((IDLE))
    IDLE -- "start=1" --> START((START))
    START -- "S issued" --> WR_ADDR((WR ADDR))
    WR_ADDR -- "ACK" --> WR_REG((WR REG))
    WR_REG -- "ACK" --> RPT_START((RPT START))
    RPT_START -- "start re-asserted or issued" --> RD_ADDR((RD ADDR))
    RD_ADDR -- "ACK" --> RD_DATA((RD DATA))
    RD_DATA -- "NACK" --> IDLE
    RD_DATA -- "byte rx" --> DECODE((DECODE))
    DECODE -- "bad key" --> ABSENT((ABSENT))
    DECODE -- "valid key" --> LOCK((LOCK))
    LOCK -- "catch" --> ABSENT
    ABSENT -- "catch" --> DONE((DONE))
    IDLE -- "retry < 3" --> IDLE
  
```

5

§7 I²C Transaction Waveform

The complete transaction to read SPD byte 0x02 from a slot-0 DIMM (device address 0x50) is shown below. The Repeated Start avoids releasing the bus between the address-phase write and the data read.

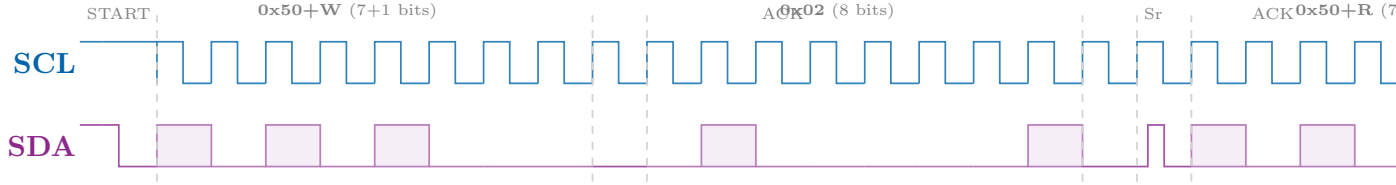


Figure 3: I²C transaction waveform for a single SPD key byte read. Hatched SDA regions indicate data bits (schematic). The Repeated Start (Sr) at cycle 19 keeps the bus locked between write and read phases.

§8 Timing & Latency Analysis

The total I²C transaction consists of the following byte transfers:

Phase	Bits	I ² C clocks
START condition	—	1
Device address + W	8	9
ACK	1	—
Register address byte	8	9
ACK	1	—
Repeated START	—	1
Device address + R	8	9
ACK	1	—
Data byte (key byte)	8	9
NACK + STOP	1	2
Total		40 I²C clocks

Table 4: Transaction breakdown. ACK bits are counted within each 9-clock byte frame.

At 400 kHz Fast-Mode, 40 SCL cycles take $40 \times 2.5 \mu\text{s} = 100 \mu\text{s}$. At a 100 MHz system clock this is exactly **10 000 system clock cycles** — still orders of magnitude faster than the $\sim 100\,000$ cycles required by a full training-based probe sequence.

Latency Comparison

Method	Cycles (100 MHz sys clk)	Wall time
SPD I ² C read (400 kHz)	~10 000	~100 μ s
SPD I ² C read (100 kHz)	~40 000	~400 μ s
Training-based probe FSM	~500 000+	~5 ms+

§9 Integration with `ddr_version_detect`

`spd_reader` sits in parallel with `ddr_version_detect` and feeds the same MC config mux. The MC top-level priority logic is:

$$\text{ddr_version} = \begin{cases} \text{spd_reader.ddd_version} & \text{if } \text{spd_valid} = 1 \\ \text{detect.ddd_version} & \text{if } \text{detect.detected} = 1 \\ 3'b111 \text{ (unknown)} & \text{otherwise (error)} \end{cases}$$

When `spd_valid` is asserted, the MC can assert `start_detect = 0` to gate off the probe FSM and save power. If `spd_absent` is asserted, the MC immediately starts the training probe to avoid wasting time waiting for an I²C timeout.

RTL Build-Out Plan

1. `spd_reader.sv` — this module (I²C master + FSM + decoder)
2. `i2c_bit_master.sv` — reusable open-drain I²C bit primitive
3. `spd_decoder.sv` — combinational key byte to `ddr_version` map
4. `ddr_version_detect.sv` — training-based fallback (existing module)
5. `ddr_mc_top.sv` — priority mux combining both detection outputs